DIPLOMADO EN FULLSTACK DEVELOPER BACKEND FRONTEND

DESARROLLO BACKEND CON DJANGO

MÉTODO DE EVALUACIÓN

Asistencia	10%
Participación en clases	10%
Examen	40%
Proyecto	40%

TABLA DE CONTENIDOS

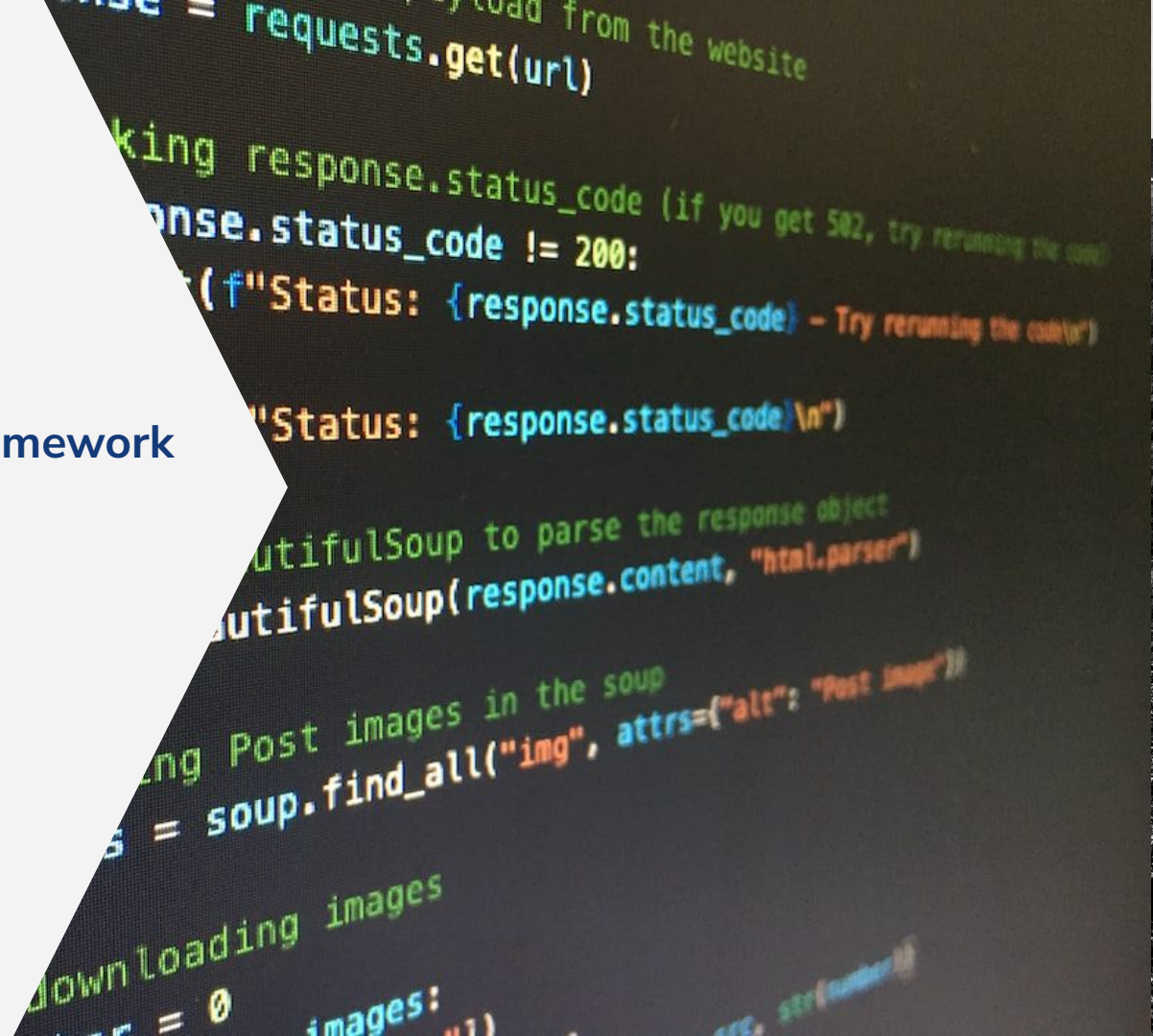
Estructura del Framework

Modelos y Administración

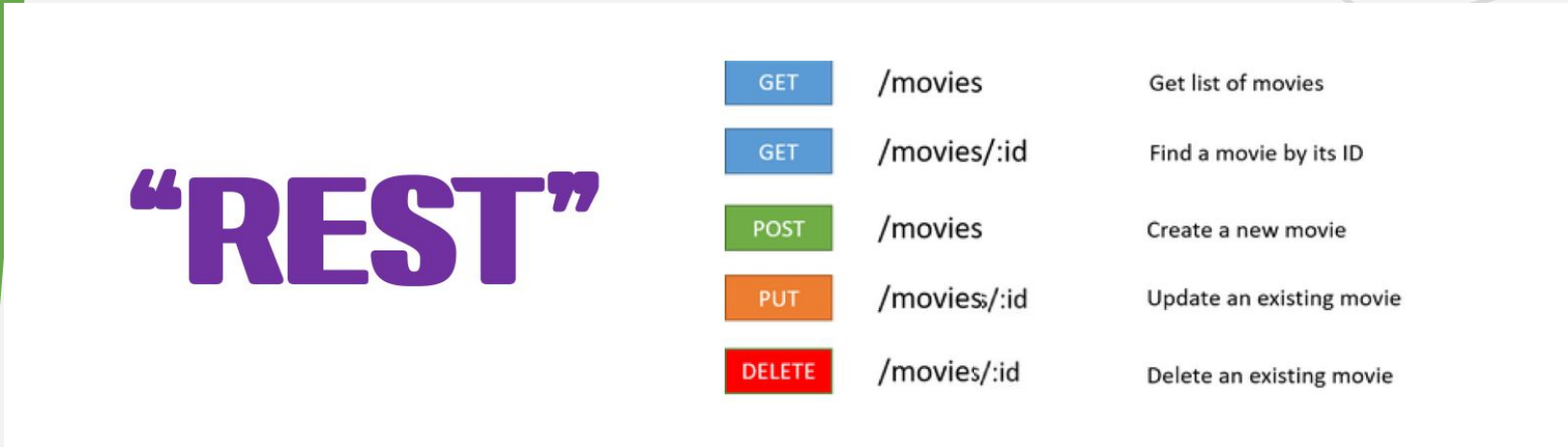
Django Rest Framework

Testing y Logs

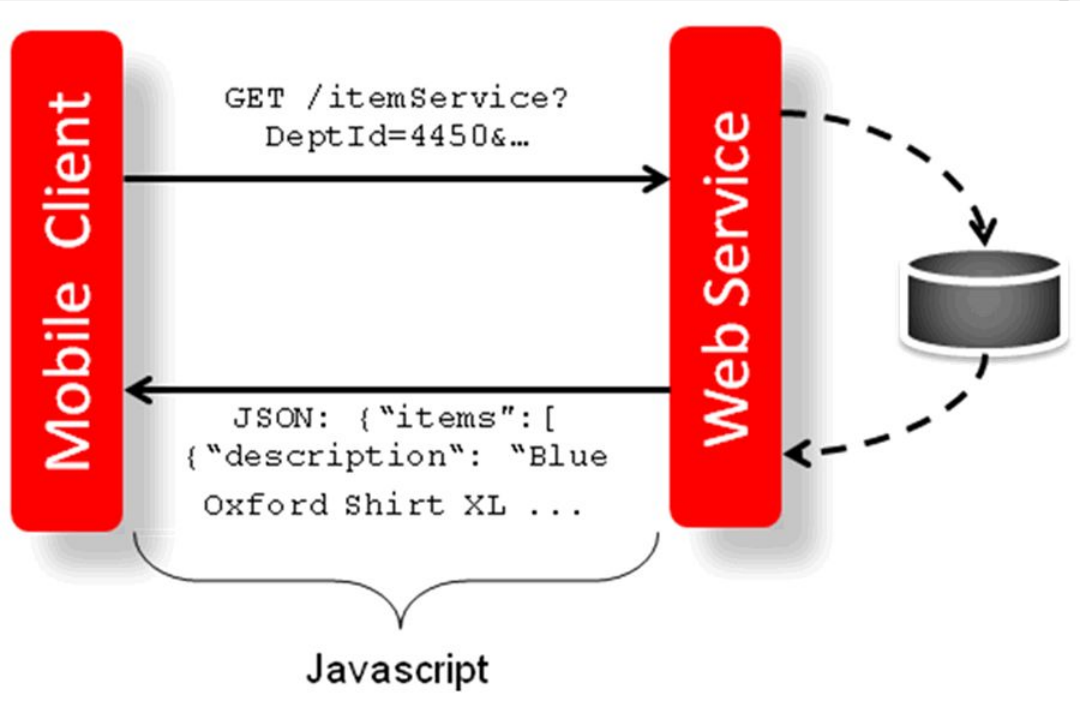
Django Rest Framework
DRF



Trabajando con API REST



Trabajando con API REST



Trabajando con API REST

CÓDIGOS DE ESTADO

- 20X(CORRECTO)
- 30X(CACHÉ - REDIRECCIONAMIENTO)
- 40X(NO ENCONTRADO)
- 50X(ERROR DE SERVIDOR)

DRF - ModelViewSet

Trabajando con ModelViewSet

La clase ModelViewSet se hereda de GenericAPIView e incluye implementaciones para varias acciones, mezclando el comportamiento de las distintas clases mixtas.

Las acciones proporcionadas por la clase ModelViewSet son .list(), .retrieve(), .create(), .update(), .partial_update() y .destroy().

https://www.django-rest-framework.org/api-guide/viewsets/#modelviewset

Instalación

```
pip install djangorestframework
```

settings.py

```
INSTALLED_APPS = [
    ...
    'rest_framework',
]

REST_FRAMEWORK = {
    #"DEFAULT_PERMISSION_CLASSES": ("rest_framework.permissions.IsAuthenticated",),
    "DEFAULT_PERMISSION_CLASSES": ("rest_framework.permissions.AllowAny",),
}
```

Serializers

inventario/serializers.py

```python
from rest_framework import serializers
from .models import Categoria


class CategoriaSerializer(serializers.ModelSerializer):
    class Meta:
        model = Categoria
        fields = "__all__"
```

Los serializadores permiten que datos complejos, como conjuntos de consultas e instancias de modelos se conviertan en tipos de datos nativos de Python que luego se pueden representar fácilmente en JSON.

Vista

inventario/views.py

```
from rest_framework import viewsets
from .models import Categoria
from .serializers import CategoriaSerializer


class CategoriaViewSet(viewsets.ModelViewSet):
    queryset = Categoria.objects.all()
    serializer_class = CategoriaSerializer
```

URLs

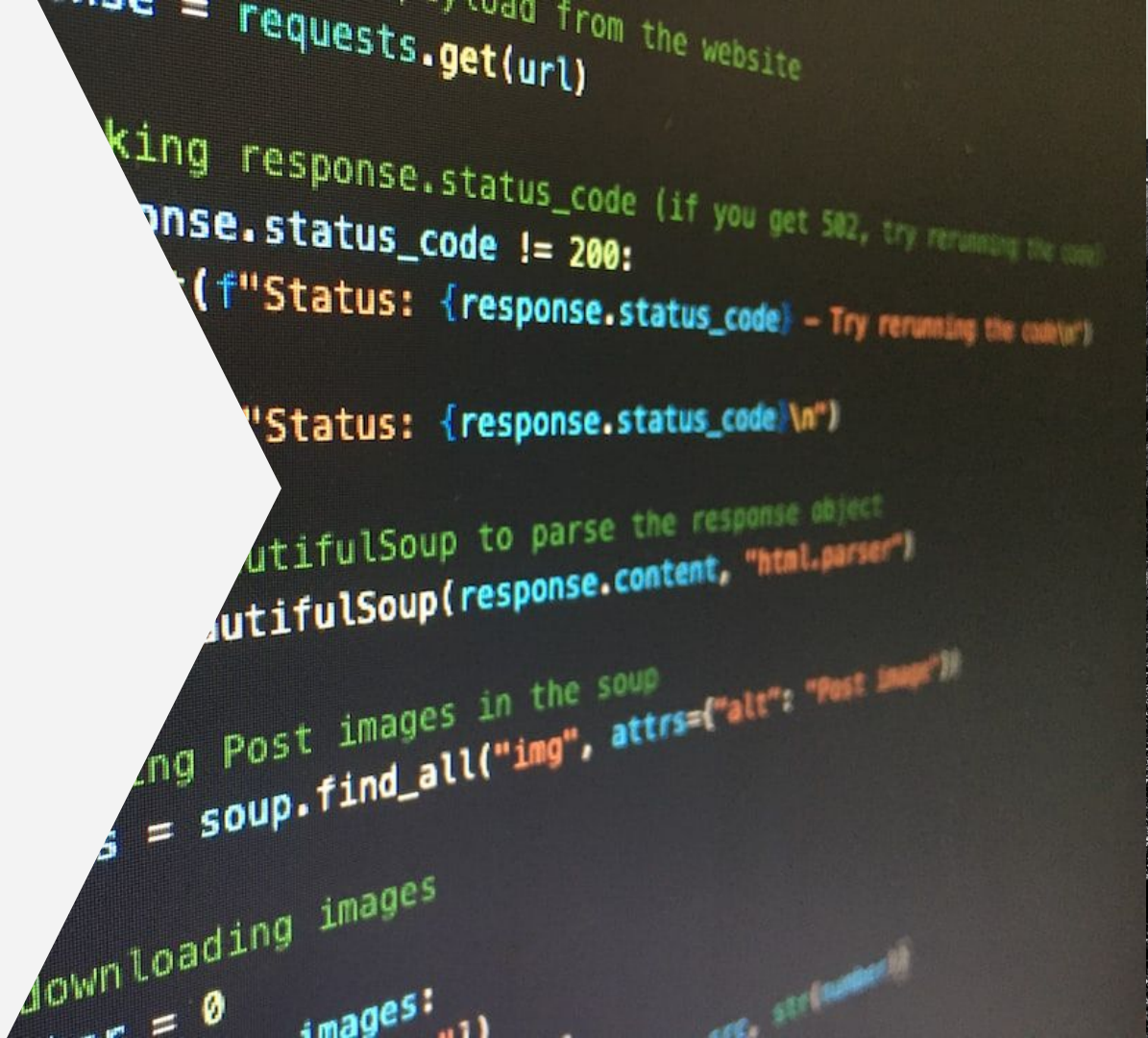
inventario/urls.py

```python
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from . import views

router = DefaultRouter()
router.register(r"categorias", views.CategoriaViewSet)


urlpatterns = [
    path('contact/<str:name>', views.contact, name='contacto'),
    # path('categorias', views.categorias, name='categorias'),
    # path('', views.index, name='index'),
    path('', include(router.urls)),
]
```

DRF - Swagger


requests.get(url)
response.status_code != 200:
Status: {response.status_code}
Status: {response.status_code}\n
BeautifulSoup(response.content, "html.parser")
soup.find_all("img", attrs={"alt": "Post image"})
downloading images

Instalación

```
pip install drf-yasg
```

https://drf-yasg.readthedocs.io/en/stable/readme.html

Configuración

settings.py

```
INSTALLED_APPS = [
    ...
    'drf_yasg',
]
```

URLs

urls.py

```python
from django.urls import re_path
from drf_yasg.views import get_schema_view
from drf_yasg import openapi
from rest_framework import permissions
```

```python
....
```

```python
schema_view = get_schema_view(
    openapi.Info(
        title="Curso de Django",
        default_version="v1",
        description="Base Django",
        terms_of_service="",
        contact=openapi.Contact(email="arteagamarcelo@gmail.com"),
        license=openapi.License(name="MIT License"),
    ),
    public=True,
    permission_classes=[permissions.AllowAny],
)

if settings.DEBUG:
    urlpatterns += [
        path("admin/", admin.site.urls),
        path(
            "swagger<format>/",
            schema_view.without_ui(cache_timeout=0),
            name="schema-json",
        ),
        path(
            "swagger/",
            schema_view.with_ui("swagger", cache_timeout=0),
            name="schema-swagger-ui",
        ),
        path("redoc/", schema_view.with_ui("redoc", cache_timeout=0), name="schema-redoc"),
    ] + static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
```

DRF - GenericAPIView

Trabajando con GenericAPIView

Las vistas genéricas de Django... se desarrollaron como un atajo para los patrones de uso común... Toman ciertos modismos y patrones comunes que se encuentran en el desarrollo de vistas y los abstraen para que pueda escribir rápidamente vistas comunes de datos sin tener que repetirlo.

http://www.tomchristie.com/rest-framework-2-docs/api-guide/generic-views#genericapiview

Creando una GenericAPIView

inventario/views.py

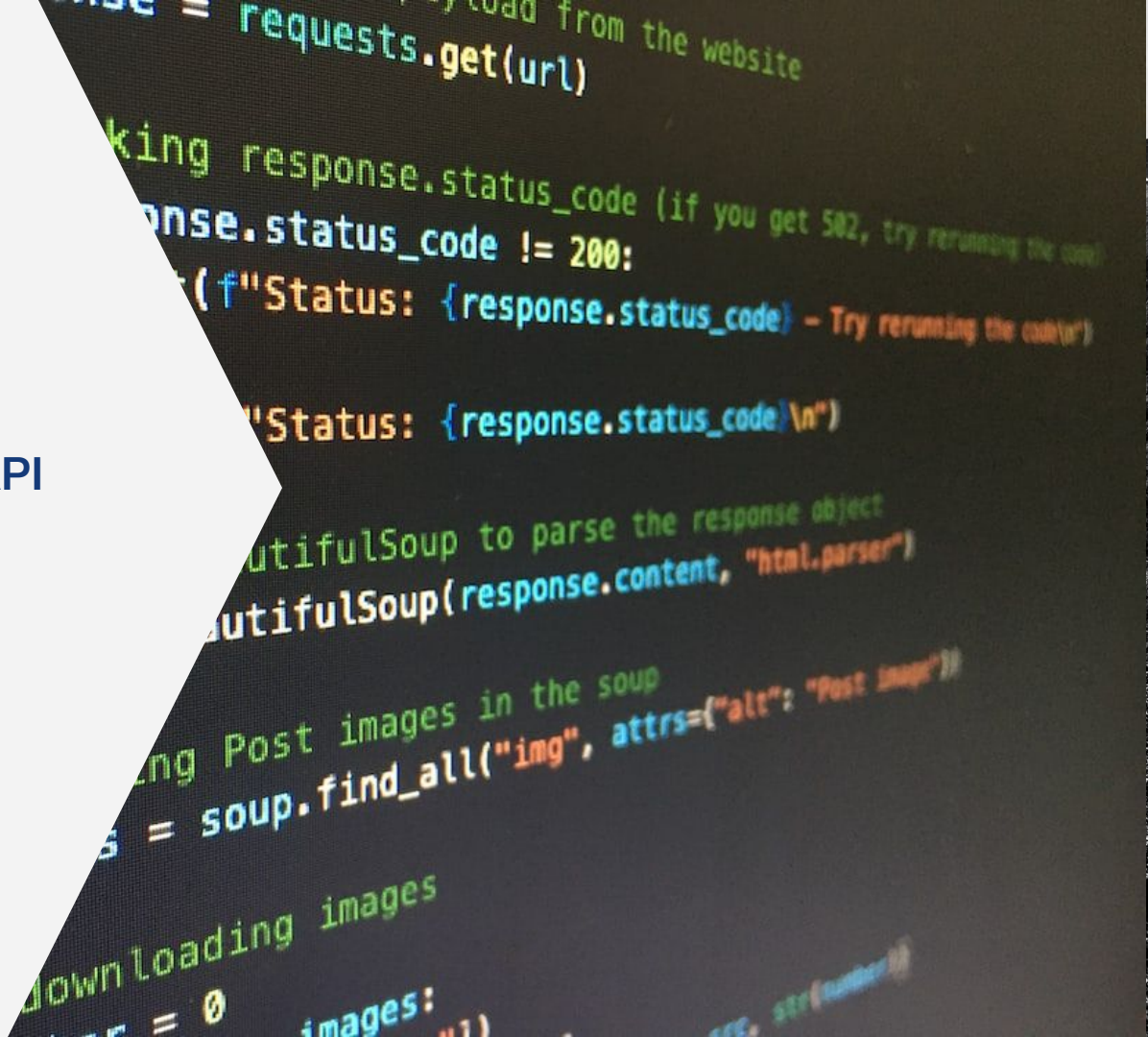
```
class CategoriaCreateView(generics.CreateAPIView, generics.ListAPIView):
    queryset = Categoria.objects.all()
    serializer_class = CategoriaSerializer
```

Creando una GenericAPIView

inventario/urls.py

```
urlpatterns = [
    ...
    path('categoria/crear/', views.CategoriaCreateView.as_view()),
    ...
]
```

DRF - Custom API



Api Personalizada basica

inventario/views.py

```python
@api_view(["GET"])
def categoria_count(request):
    """
    Cantidad de items, ACCESS ROL: ANONYMOUS
    Cantidad de items registrados en categoria
    """

    try:
        cantidad = Categoria.objects.count()
        return JsonResponse(
            {
                "cantidad": cantidad
            },
            safe=False,
            status=200,
        )
    except Exception as e:
        return JsonResponse({"message": str(e)}, status=400)
```

safe=False, permite a JsonResponse recibir información como Diccionarios y otros.
safe=True, solo permite Diccionarios.
Ex: [{“demo”:1}] solo funcionara con safe=False por que es un tipo list

Api Personalizada basica

inventario/urls.py

```
urlpatterns = [
    ...
    path('categorias/cantidad/', views.categoria_count),
    ...
]
```

Utilizando ModelSerializers para el Output

inventario/serializers.py

```
from .models import Producto
```

```
class ProductoSerializer(serializers.ModelSerializer):
    class Meta:
        model = Producto
        fields = "__all__"
```

Utilizando ModelSerializers para el Output

inventario/views.py

```python
@api_view(["GET"])
def productos_en_unidades(request):
    """
    Lista de productos en unidades, ACCESS ROL: ANONYMOUS
    Listado de productos que son filtrados por unidad
    """

    try:
        productos = Producto.objects.filter(unidades='u')
        return JsonResponse(
            ProductoSerializer(productos, many=True).data,
            safe=False,
            status=200,
        )
    except Exception as e:
        return JsonResponse({"message": str(e)}, status=400)
```

Utilizando ModelSerializers para el Output

inventario/urls.py

```
urlpatterns = [
    ...
    path('productos/filtrar/unidades/', views.productos_en_unidades),
    ...
]
```

Utilizando Custom Serializers para el Output

inventario/serializers.py

```
class ReporteProductosSerializer(serializers.Serializer):
    cantidad = serializers.IntegerField()
    productos = ProductoSerializer(many=True)
```

Utilizando Custom Serializers para el Output

inventario/views.py

```python
@api_view(["GET"])
def reporte_productos(request):
    """
    Lista de productos en unidades, ACCESS ROL: ANONYMOUS
    Listado de productos que son filtrados por unidad
    """

    try:
        productos = Producto.objects.filter(unidades='u')
        cantidad = productos.count()
        return JsonResponse(
            ReporteProductosSerializer({"cantidad": cantidad,
                                        "productos": productos}).data,
            safe=False,
            status=200,
        )
    except Exception as e:
        return JsonResponse({"message": str(e)}, status=400)
```

Utilizando Custom Serializers para el Output

inventario/urls.py

```
urlpatterns = [
    ...
    path('reporte/productos/', views.reporte_productos),
    ...
]
```

Utilizando Custom Serializers para el Input

inventario/validators.py

```
def validar_nombre(value):
    if value == "Comida":
        raise ValidationError(
            '%(value)s no es un texto permitido',
            params={'value': value},
        )
```

Utilizando Custom Serializers para el Input

inventario/serializers.py

```
from .validators import validar_nombre_comida


class ContactSerializer(serializers.Serializer):
    email = serializers.EmailField()
    subject = serializers.CharField(max_length=100,
                                    validators=[validar_nombre_comida,])
    body = serializers.CharField(max_length=255)
```

Utilizando Custom Serializers para el Input

inventario/views.py

```
@api_view(["POST"])
def enviar_mensaje(request):
    """
    Enviar mensajes, ACCESS ROL: ANONYMOUS
    Envia mensajes a un destinatario via email
    """
    cs = ContactSerializer(data=request.data)
    if cs.is_valid():
        return JsonResponse({"message": "Mensaje enviado correctamente"}, status=20
    else:
        return JsonResponse({"mensaje": cs.errors}, status=200)
```

Utilizando Custom Serializers para el Input

inventario/urls.py

```
urlpatterns = [
    ...
    path('reporte/productos/', views.reporte_productos),
    ...
]
```

GRACIAS